PROJECT REPORT



Intelligent Document Assistant:
A Retrieval-Augmented Generation (RAG) System

Project ID: PRJ-2026-004

Team ID: RAG-MIND MESH AI

Team GitHub Repository: https://github.com/chakalihitesh/GENAI_PROJECT.git

Student Name	Registration Number	Individual GitHub Link
C SINDHU	99240040807	https://github.com/SindhuKamakshi/GENAI-RAG-PROJECT-CODEBASE-AI.git
VELMALA MOURYA RAJ	99240040885	https://github.com/manumouryavelmala-cell/GenAI-project
HARSHITH CH	99240040213	https://github.com/harshith3987-lgtm/Gen_AI
CHAKALI HITESH	99240040870	https://github.com/chakalihitesh/GENAI_PROJECT.git

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

ABSTRACT

The Intelligent Document Assistant is an AI-based application designed to help users find information in academic and institutional documents. Instead of manually searching lengthy documents, users can ask questions in natural language and receive responses based on the content of the uploaded files. The system uses Retrieval-Augmented Generation (RAG), combining semantic retrieval with language-model response generation.

During document ingestion, text is extracted from PDF or TXT files, divided into smaller chunks, converted into embeddings, and stored in a vector database. When a question is submitted, the system retrieves relevant chunks and supplies them as context to a language model. The model then generates an answer grounded in the retrieved content. The described technology stack includes Python, LangChain, ChromaDB, HuggingFace embeddings, and Llama 3 accessed through the Groq API.

Keywords: RAG, semantic search, embeddings, vector database, LangChain, ChromaDB, LLM.

TABLE OF CONTENTS

No.	Section
1	Introduction
2	Problem Statement and Objectives
3	System Architecture (RAG Pipeline)
4	Implementation Details and Technology Stack
5	Sample Chunking Configuration
6	Results and Testing
7	Advantages and Limitations
8	Future Scope
9	Conclusion
10	References

1. INTRODUCTION

Educational institutions maintain a large amount of information in documents such as academic handbooks, course catalogs, examination regulations, attendance policies, and administrative circulars. Students and faculty members often need to locate specific information in these documents. Searching manually can take time, especially when the information is spread across many pages or sections.

Traditional keyword searching may not find useful results when a question is phrased differently from the wording in the document. The Intelligent Document Assistant addresses this problem by allowing users to ask questions in natural language. It searches the uploaded documents and uses the retrieved information to prepare an answer.

The system uses Retrieval-Augmented Generation (RAG). First, documents are processed and stored in a searchable form. Later, when a question is asked, relevant document sections are retrieved and provided to a language model as context. This helps make responses more relevant to the supplied documents.

The project combines document processing, semantic search, and AI-based response generation to make institutional information easier to access.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Students and staff may need to search lengthy institutional documents to find details such as attendance requirements, examination rules, course information, or academic regulations. Manual reading is time-consuming, while keyword search may miss relevant passages when different wording is used. A system is therefore needed to retrieve document information through natural-language questions and present it in a clear form.

2.2 Objectives

• Develop an assistant that answers questions using uploaded academic or institutional documents.

• Allow users to search documents using natural-language questions.

• Retrieve relevant passages through semantic similarity search.

• Generate responses using retrieved document context.

• Reduce the time and effort required for manual document searching.

• Help limit unsupported responses by grounding answers in source material.

3. SYSTEM ARCHITECTURE (RAG PIPELINE)

The system has two main phases: Data Ingestion (offline) and Query Processing and Generation (runtime). The first phase prepares documents for retrieval. The second phase uses the stored information to answer user questions.

3.1 Data Ingestion Phase (Offline)

• **Document upload:** Institutional documents are provided in PDF or TXT format.

• **Text extraction:** A document loader and parser extract the textual content.

• **Text splitting:** The extracted text is divided into smaller chunks using a recursive character splitter.

• **Embedding generation:** The embedding model converts each chunk into a numerical vector.

• **Vector storage:** Chunks, embeddings, and associated metadata are stored in ChromaDB.

3.2 Query Processing and Generation Phase (Runtime)

• **User query:** The user submits a question in natural language.

• **Query embedding:** The same embedding model converts the question into a vector.

• **Semantic retrieval:** ChromaDB retrieves the most relevant chunks using vector similarity (top-K retrieval).

• **Prompt construction:** Retrieved context, the question, and system instructions are combined.

• **Answer generation:** Llama 3, accessed through the Groq API, generates a response using the supplied context.

• **Response:** The application presents the answer, with source citations where supported by the implementation.

3.3 Workflow Summary

Document Upload → Text Extraction → Chunking → Embeddings → ChromaDB Storage → User Query → Semantic Search → Context + Query Prompt → Llama 3 via Groq → Answer

The diagram and workflow describe the architecture supplied for this report. Actual behavior depends on the implementation and configuration deployed with the application.

4. IMPLEMENTATION DETAILS AND TECHNOLOGY STACK

The system combines a Python backend, a RAG orchestration framework, a vector database, an embedding model, and a language model API.

Component	Technology Used	Purpose
Programming Language	Python 3.10	Implements backend logic and document-processing steps.
Orchestration Framework	LangChain	Connects document loaders, text splitters, embedding models, vector stores, and model calls.
Vector Database	ChromaDB	Stores and retrieves document embeddings and associated text.
Embedding Model	HuggingFace all-MiniLM-L6-v2	Converts document chunks and user questions into vector representations.
Large Language Model	Llama 3 via Groq API	Generates natural-language responses from retrieved context.

4.1 Python

Python is used to implement the backend workflow and integrate document parsing, chunking, embeddings, retrieval, and language-model calls.

4.2 LangChain

LangChain provides components for organizing the RAG pipeline, including document loaders, text splitters, embedding interfaces, and vector-store integration.

4.3 ChromaDB

ChromaDB stores vector representations alongside document text and metadata. At query time, it supports similarity-based retrieval of relevant chunks.

4.4 Embeddings and Language Model

The all-MiniLM-L6-v2 model represents text as vectors for semantic comparison. Llama 3 is used to generate the final response, with the Groq API providing model inference.

5. SAMPLE CHUNKING CONFIGURATION

Chunking divides long text into smaller sections so that the retrieval system can search for focused passages. The project configuration uses LangChain's Recursive Character Text Splitter with a target chunk size of 1000 characters and an overlap of 200 characters.

Sample configuration:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
chunk_size=1000, chunk_overlap=200,
length_function=len
)
```

Parameter	Value	Meaning
Chunk size	1000 characters	Target maximum length of each text segment.
Chunk overlap	200 characters	Repeats part of the preceding chunk to preserve continuity.
Length function	len	Measures the text length in characters.

For example, if one chunk covers characters 1–1000, the next chunk begins around character 801. The shared 200 characters help retain context when a sentence or idea crosses a chunk boundary.

Chunking can improve retrieval focus and reduce unnecessary context. The best chunk size may vary with document structure and should be validated using representative questions.

6. RESULTS AND TESTING

Testing should check whether the system retrieves the right document passages and produces answers consistent with those passages. Representative questions may cover attendance, grading, credit requirements, examinations, registration, and graduation rules.

Suggested test cases:

Test	Expected behavior
Ask a question answered directly in a document	Retrieve the relevant passage and answer consistently with it.
Ask a question using different wording	Find semantically relevant content even without an exact keyword match.
Ask about information absent from the documents	Indicate that the answer cannot be found in the supplied content.
Ask for source files or citations	Identify available source information accurately, if supported by the implementation.

Any numerical performance claims, such as response time or hallucination reduction, should be included only when supported by actual test records. No independently verified benchmark results are assumed in this report.

7. ADVANTAGES AND LIMITATIONS

7.1 Advantages

• Provides a natural-language interface for searching documents.

• Uses semantic retrieval rather than exact keyword matching alone.

• Can reduce manual effort when locating information in lengthy documents.

• Separates document preparation, retrieval, and answer generation into manageable stages.

7.2 Limitations

• Answer quality depends on the accuracy and completeness of the uploaded documents.

• Relevant information may be missed if retrieval does not select the correct chunks.

• A language model can still produce errors; RAG does not guarantee correctness.

• Model API availability, network latency, and deployment configuration can affect response time.

8. FUTURE SCOPE

• Support additional file types and improve extraction from tables, images, and scanned documents.

• Add user authentication and role-based access to restricted documents.

• Provide feedback options so users can flag incomplete or incorrect answers.

• Evaluate improved retrieval methods, including reranking and query refinement.

• Add document versioning so answers can be tied to the correct edition of a policy.

9. CONCLUSION

The Intelligent Document Assistant applies Retrieval-Augmented Generation to make information in academic and institutional documents easier to find. Documents are extracted, split into chunks, embedded, and stored in a vector database. At runtime, the system retrieves relevant passages and supplies them to a language model to generate a response.

The approach combines semantic search with natural-language generation and can reduce the effort involved in manual document searching. Its reliability depends on document quality, retrieval performance, and careful validation of generated answers. The project provides a foundation that can be extended with stronger retrieval, broader document support, and access controls.

10. REFERENCES

• LangChain documentation: https://python.langchain.com/

• Chroma documentation: https://docs.trychroma.com/

• Hugging Face model page: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

• Groq API documentation: https://console.groq.com/docs

Note: Replace the cover-page placeholders with your details. Confirm the exact library versions and any measured results against your code and test records before submission.